

Phase 3: Linux Systems & Networking

You now have a production-grade Thread Pool. But right now, it has nothing to do. In competitive programming, data comes from `std::cin`. In a distributed system, data comes from across the globe through **Network Sockets**.

Since you are running macOS, you are in a Unix-like environment. This means you will use the POSIX standard API to talk to the operating system.

Week 1: "Everything is a File" & Socket Programming

This week, we bridge the gap between your C++ logic and the Operating System's network stack.

Day 1: File Descriptors (The OS Currency)

In Unix systems (including macOS and Linux), *everything is a file*. A text document on your SSD is a file. Your keyboard is a file. A network connection to a server in Japan is a file.

When you open any of these, the OS gives your C++ program a **File Descriptor (FD)**. It is simply an integer (like 3, 4, or 12).

- FD 0 is standard input (`cin`).
- FD 1 is standard output (`cout`).
- FD 2 is standard error (`cerr`).
- Any new file or network connection you open will be assigned FD 3, 4, 5, etc.

Day 2 & 3: The TCP Socket Lifecycle

To create a backend server, you must ask the OS to open a special file descriptor called a **Socket** and bind it to a port on your machine (e.g., port 8080).

Here is the exact lifecycle of a TCP Server:

1. `socket()`: "Hey OS, give me a file descriptor for an IPv4 TCP network connection."
2. `bind()`: "Attach this file descriptor to Port 8080 on my machine."
3. `listen()`: "Start monitoring Port 8080 for incoming connections."
4. `accept()`: "Wait (block) until a user actually connects, then give me a *new* file descriptor specifically for talking to that user."

Here is the raw, POSIX C++ code to achieve this. It looks ugly compared to modern C++ because this API was written in C in the 1980s. But every web framework on Earth (including FastAPI and Node.js) runs this exact code under the hood.

C++

```
#include <iostream>
```

```

#include <string>
#include <unistd.h>    // For close(), read(), write() on macOS/Linux
#include <sys/socket.h> // For socket(), bind(), listen(), accept()
#include <netinet/in.h> // For sockaddr_in
#include <arpa/inet.h> // For htons()

int main() {
    // 1. Create the socket (IPv4, TCP)
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        std::cerr << "Failed to create socket.\n";
        return 1;
    }

    // 2. Configure the Port and IP address
    struct sockaddr_in address;
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY; // Listen on all network interfaces
    address.sin_port = htons(8080);      // Port 8080 (htons handles Endianness)

    // 3. Bind the socket to the port
    if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) < 0) {
        std::cerr << "Bind failed. Port might be in use.\n";
        close(server_fd);
        return 1;
    }

    // 4. Start listening (allow a backlog of 10 pending connections)
    if (listen(server_fd, 10) < 0) {
        std::cerr << "Listen failed.\n";
        close(server_fd);
        return 1;
    }

    std::cout << "[Server] Listening on port 8080...\n";

    // 5. Accept an incoming connection (THIS BLOCKS THE THREAD UNTIL SOMEONE
    CONNECTS)
    int addrlen = sizeof(address);
    int client_fd = accept(server_fd, (struct sockaddr*)&address, (socklen_t*)&addrlen);
    if (client_fd < 0) {
        std::cerr << "Accept failed.\n";
        close(server_fd);
        return 1;
    }
}

```

```

std::cout << "[Server] A client connected! Client FD: " << client_fd << "\n";

// 6. Read data from the client
char buffer[1024] = {0};
ssize_t bytes_read = read(client_fd, buffer, sizeof(buffer) - 1);
std::cout << "Received: " << buffer << "\n";

// 7. Send a response back
std::string response = "Hello from your C++ Backend!\n";
write(client_fd, response.c_str(), response.length());

// 8. Close the connections
close(client_fd);
close(server_fd);
std::cout << "[Server] Connection closed. Shutting down.\n";

return 0;
}

```

Day 4: Talking to Your Server

If you compile and run the code above, the program will freeze at the `accept()` line. It is waiting for a client.

To test it, you don't need to write a C++ client yet. Open a second terminal window on your macOS and use `nc` (Netcat), which is a built-in network testing tool:

Bash

```
nc localhost 8080
```

Type any message into the Netcat terminal and press Enter. You will see it appear in your C++ server terminal, and your Netcat terminal will receive the "Hello from your C++ Backend!" response.

Day 5: The "Blocking" Bottleneck

Did you notice the critical flaw in the server code above?

It calls `accept()`, talks to *one* client, and then shuts down. If you put `accept()` inside a `while(true)` loop, you can talk to multiple clients. However, `read()` is a **blocking** call. If Client 1 connects but sends no data, the entire server freezes waiting for Client 1. If Client 2 tries to connect during this time, they are locked out.

To solve this, we must pass the `client_fd` to the **Thread Pool** we built in Phase 2!

The Weekend Assignment: "The Multithreaded Echo Server"

This is the first true backend architecture you will build.

Task: Combine your Phase 2 `ThreadPool.hpp` with the Socket code above.

1. Boot up your `ThreadPool` with 4 workers.
2. The main thread runs an infinite `while(true)` loop, doing nothing but calling `accept()`.
3. The moment `accept()` returns a valid `client_fd`, the main thread **submits** a task to the Thread Pool to handle that specific file descriptor.
4. The worker thread runs a `while(true)` loop reading from the `client_fd`. Whatever text the client sends, the worker thread should capitalize every letter and `write()` it back to the client.
5. If the client types "QUIT" or disconnects, the worker thread calls `close(client_fd)` and finishes the task, returning to the pool to wait for the next user.

Testing it:

Run the server. Open three different terminal windows and run `nc localhost 8080` in all three. You should be able to type in all three terminals simultaneously, proving that your C++ server is handling concurrent network connections like a real web server.

This is the moment your C++ code stops being a local script and becomes a true backend server.

We are going to combine the POSIX Network Sockets with your preferred `ThreadPool`.

The Architecture: Acceptor vs. Workers

In a high-performance server, you must ruthlessly protect the **Main Thread**.

- The **Main Thread** has only one job: wait for `accept()` to return a new `client_fd`, toss it into the Thread Pool, and immediately go back to waiting for the next user. It *never* reads or writes data.
- The **Worker Threads** (inside your pool) do the heavy lifting. They wake up, take the `client_fd`, read the data, process it (e.g., convert to uppercase), write the response, and finally close the connection.

The Code: Multithreaded Echo Server

Here is the complete `main.cpp`. It includes your exact `ThreadPool` class and the socket logic.

Compile this on your macOS using `g++ -std=c++20 main.cpp`.

C++

```
#include <iostream>
#include <string>
#include <vector>
```

```

#include <thread>
#include <queue>
#include <mutex>
#include <condition_variable>
#include <functional>
#include <algorithm>
#include <cctype>

// Unix Socket Headers
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>

// --- Your ThreadPool Implementation ---
class ThreadPool {
private:
    std::vector<std::thread> workers;
    std::queue<std::function<void()>> tasks;
    std::mutex queue_mutex;
    std::condition_variable condition;
    bool stop = false;

public:
    ThreadPool(size_t num_threads) {
        for(size_t i = 0; i < num_threads; ++i) {
            workers.emplace_back([this] {
                while(true) {
                    std::function<void()> task;
                    {
                        std::unique_lock<std::mutex> lock(this->queue_mutex);
                        this->condition.wait(lock, [this] {
                            return this->stop || !this->tasks.empty();
                        });

                        if(this->stop && this->tasks.empty()) return;

                        task = std::move(this->tasks.front());
                        this->tasks.pop();
                    }
                    task();
                }
            });
        }
    }

    void enqueue(std::function<void()> task) {

```

```

    {
        std::unique_lock<std::mutex> lock(queue_mutex);
        tasks.push(std::move(task));
    }
    condition.notify_one();
}

~ThreadPool() {
    {
        std::unique_lock<std::mutex> lock(queue_mutex);
        stop = true;
    }
    condition.notify_all();
    for(std::thread &worker: workers) {
        worker.join();
    }
}

};

// --- The Worker Logic ---
// This function runs entirely inside a background worker thread
void handle_client(int client_fd) {
    char buffer[1024] = {0};

    // Send a welcome message
    std::string welcome = "Welcome to the C++ ThreadPool Server! Type 'QUIT' to exit.\n";
    write(client_fd, welcome.c_str(), welcome.length());

    while (true) {
        // Clear the buffer
        memset(buffer, 0, sizeof(buffer));

        // Read data from the client (Blocks until the client types something)
        ssize_t bytes_read = read(client_fd, buffer, sizeof(buffer) - 1);

        // If read returns 0 (client disconnected) or -1 (error)
        if (bytes_read <= 0) {
            std::cout << "[Worker " << std::this_thread::get_id() << "] Client disconnected.\n";
            break;
        }

        std::string received(buffer);

        // Check for exit command
        if (received.find("QUIT") != std::string::npos) {
            std::string goodbye = "Goodbye!\n";

```

```

        write(client_fd, goodbye.c_str(), goodbye.length());
        std::cout << "[Worker " << std::this_thread::get_id() << "] Client gracefully quit.\n";
        break;
    }

    // Process data: Convert to uppercase
    std::transform(received.begin(), received.end(), received.begin(), ::toupper);
    std::string response = "SERVER ECHO: " + received;

    // Send back to client
    write(client_fd, response.c_str(), response.length());
}

// CRITICAL: Always close the file descriptor when done, or you will leak sockets!
close(client_fd);
}

// --- The Main Acceptor Loop ---
int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        std::cerr << "Failed to create socket.\n";
        return 1;
    }

    // Prevent "Address already in use" error if you restart the server quickly
    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in address;
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(8080);

    if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) < 0) {
        std::cerr << "Bind failed.\n";
        return 1;
    }

    if (listen(server_fd, 10) < 0) {
        std::cerr << "Listen failed.\n";
        return 1;
    }

    std::cout << "--- Server Booted ---\n";
    std::cout << "Listening on port 8080 with 4 Worker Threads...\n";
}

```

```

ThreadPool pool(4);

while (true) {
    int addrlen = sizeof(address);
    // MAIN THREAD BLOCKS HERE: Waiting for a new user
    int client_fd = accept(server_fd, (struct sockaddr*)&address, (socklen_t*)&addrlen);

    if (client_fd < 0) {
        std::cerr << "Failed to accept client.\n";
        continue;
    }

    std::cout << "[Main] New connection accepted! FD: " << client_fd << ". Handing to pool...\n";

    // CRITICAL: Capture client_fd BY VALUE [client_fd], not by reference [&]!
    // If you capture by reference, the main thread will overwrite it on the next loop iteration.
    pool.enqueue([client_fd]() {
        handle_client(client_fd);
    });
}

close(server_fd);
return 0;
}

```

How to Test Your Server Like a Pro

Do not use a web browser to test this yet (browsers speak HTTP, which is a complex text protocol we haven't implemented yet; they will send a massive GET request and get confused by your uppercase echo).

Instead, use macOS's built-in TCP tool, `nc` (Netcat).

1. **Terminal 1:** Run your server (`./a.out`).
2. **Terminal 2:** Run `nc localhost 8080`. You will see the welcome message.
3. **Terminal 3:** Run `nc localhost 8080`.
4. **Terminal 4:** Run `nc localhost 8080`.

Type messages into Terminals 2, 3, and 4 simultaneously. You will see the server effortlessly handle all of them at the exact same time, echoing your text back in uppercase, while printing the unique thread IDs in Terminal 1.

Week 2: The HTTP Protocol

Right now, your C++ server speaks raw TCP. If you point Google Chrome to `http://localhost:8080`, Chrome will connect, but it will get confused by your "Welcome" message and drop the connection.

When you previously used frameworks like FastAPI to serve your Large Language Model tools, the framework hid all of this complexity from you. It automatically unpacked the network bytes, found the requested URL, and routed it to your Python functions. Now, you are going to build that exact parsing engine yourself from scratch in C++.

Day 1: HTTP is Just Plain Text

The HTTP protocol is not a magical binary format. It is literally just a standardized formatting of text strings sent over the TCP socket you just built.

When Google Chrome connects to your server and you type `http://localhost:8080/data`, Chrome automatically sends this exact string to your C++ `read()` function:

Plaintext

```
GET /data HTTP/1.1\r\n
Host: localhost:8080\r\n
User-Agent: Mozilla/5.0\r\n
Accept: text/html\r\n
\r\n
```

Note the `\r\n` (Carriage Return + Line Feed). This is the mandatory HTTP line ending. The double `\r\n\r\n` tells the server: "I am done sending headers, the request is finished."

Day 2: The HTTP Response

If you want Chrome to display a webpage or return JSON data for your AI agents, your server cannot just send back "Hello". It must send a formally structured HTTP Response.

It looks like this:

Plaintext

```
HTTP/1.1 200 OK\r\n
Content-Type: text/html\r\n
Content-Length: 46\r\n
\r\n
<html><body><h1>C++ Backend!</h1></body></html>
```

Day 3 & 4: Upgrading the Worker Thread

As a competitive programmer, parsing strings is light work for you. We need to upgrade the `handle_client` function from last week. Instead of converting text to uppercase, it needs to:

1. Read the HTTP request.
2. Extract the HTTP Method (GET) and the Route (/ or /api).
3. Construct the HTTP Response string.
4. Send it back and close the connection.

Replace the `handle_client` function in your previous code with this new version:

C++

```
#include <sstream>
```

```
void handle_client(int client_fd) {
    char buffer[4096] = {0};

    // 1. Read the HTTP Request from the browser
    ssize_t bytes_read = read(client_fd, buffer, sizeof(buffer) - 1);
    if (bytes_read <= 0) {
        close(client_fd);
        return;
    }

    std::string request(buffer);
    std::cout << "[Worker " << std::this_thread::get_id() << "] Received Request:\n"
        << request.substr(0, request.find("\r\n")) << "\n"; // Print only the first line

    // 2. Extremely basic parsing (Find the requested path)
    std::istringstream iss(request);
    std::string method, path, protocol;
    iss >> method >> path >> protocol;

    // 3. Routing (The C++ version of @app.get("/path"))
    std::string body;
    std::string content_type = "text/html";
    std::string status_line = "HTTP/1.1 200 OK\r\n";

    if (path == "/") {
        body = "<html><body style='font-family: sans-serif;'>"
            "<h1>Welcome to the C++ High-Performance Server!</h1>"
            "<p>Served concurrently by Thread: " +
            std::to_string(std::hash<std::thread::id>{}(std::this_thread::get_id())) +
            "</p></body></html>";
    }
    else if (path == "/api/status") {
        // Simulating a JSON API endpoint
        content_type = "application/json";
        body = "{\"status\": \"healthy\", \"active_threads\": 4}";
    }
}
```

```

}
else {
    // Handle 404 Not Found
    status_line = "HTTP/1.1 404 Not Found\r\n";
    body = "<html><body><h1>404 - Page Not Found</h1></body></html>";
}

// 4. Construct the formal HTTP Response
std::string response = status_line +
    "Content-Type: " + content_type + "\r\n" +
    "Content-Length: " + std::to_string(body.length()) + "\r\n" +
    "Connection: close\r\n" +
    "\r\n" +
    body;

// 5. Send to Chrome and close
write(client_fd, response.c_str(), response.length());
close(client_fd);
}

```

Day 5: Testing Your Web Server

1. Compile and run your server.
2. Open Google Chrome.
3. Type `http://localhost:8080/` in the address bar. You should see your HTML page rendered! Refresh it fast, and you might see the Thread ID change as different workers in your pool handle the request.
4. Type `http://localhost:8080/api/status`. You will see the raw JSON output.
5. Type `http://localhost:8080/random`. You will get your custom 404 page.

The Weekend Assignment: "The HTTP Router"

Right now, our routing logic is just a giant `if-else` block inside the worker thread. If you add 50 endpoints, this becomes unmaintainable.

Your Task:

Build a `Router` class that maps URL strings to C++ functions, mimicking modern web frameworks.

1. Create a `Router` class.
2. It should have a map: `std::unordered_map<std::string, std::function<std::string()>> routes;`
3. Add a method `void add_route(std::string path, std::function<std::string()> handler)`.
4. Modify your `main()` to register routes before starting the server:

5. C++

```
Router router;
router.add_route("/api/ping", []() {
    return "{\"message\": \"pong\"}";
});
```

6.

7.

8. Pass a reference of this `Router` to your `handle_client` worker so it can look up the requested path in the map and execute the assigned lambda function in $O(1)$ time.

This project bridges the gap between low-level sockets and high-level software design.

Phase 3, Week 3: The C10k Problem & Event Loops.

You now have a multi-threaded HTTP server. But it has a fatal flaw.

Imagine you boot your server with 4 worker threads.

1. 4 clients connect to your server.
2. They open the connection, but they have slow internet, so they don't send their HTTP request immediately.
3. Your 4 worker threads call `read(client_fd)` and **block** (freeze) waiting for data.
4. A 5th client tries to connect. They are completely locked out because your entire Thread Pool is stuck waiting for the first 4 clients to send something.

If you scale this up, how does a server handle 10,000 concurrent connections (The C10k Problem) when it only has 8 CPU cores? You cannot spawn 10,000 threads.

The answer is **Non-Blocking I/O** and **I/O Multiplexing**. This is the exact underlying architecture of Redis, Nginx, and Node.js.

Day 1: Non-Blocking Sockets

By default, network sockets are "blocking". If there is no data to read, `read()` puts the thread to sleep.

We need to tell the Operating System: *"If there is no data, don't put my thread to sleep. Just return an error immediately so I can go do other things."*

We do this using the POSIX `fcntl` (File Control) command.

C++

```
#include <fcntl.h>
```

```
void make_socket_non_blocking(int socket_fd) {  
    // Get current flags  
    int flags = fcntl(socket_fd, F_GETFL, 0);  
    // Set the O_NONBLOCK flag  
    fcntl(socket_fd, F_SETFL, flags | O_NONBLOCK);  
}
```

If you call `read()` on this socket and it's empty, `read()` instantly returns `-1` and sets the global error variable `errno` to `EWOULDBLOCK` or `EAGAIN`.

Day 2 & 3: I/O Multiplexing (poll and epoll)

If sockets are non-blocking, how do we know when data *does* arrive?

Bad Idea: Run an infinite `while` loop over all 10,000 sockets calling `read()` on every single one repeatedly. (This burns 100% CPU).

Good Idea: Ask the Operating System to monitor all 10,000 sockets for you, and put your main thread to sleep until *at least one* of them has data.

The POSIX standard way to do this is the `poll()` system call (or `epoll` on Linux, `kqueue` on macOS). Since you are developing on macOS, `poll()` is universally supported and the best starting point.

C++

```
#include <poll.h>
```

```
#include <vector>
```

```
#include <iostream>
```

```
// ... inside your main thread ...
```

```
// Create a vector of "pollfd" structures. This tells the OS what to watch.
```

```
std::vector<struct pollfd> fds;
```

```
// Add our main server socket to watch for new incoming connections (POLLIN = data ready to read)
```

```
fds.push_back({server_fd, POLLIN, 0});
```

```
while (true) {
```

```
    // 1. Sleep until the OS tells us at least one socket is active
```

```
    // The '-1' means wait forever (no timeout)
```

```
    int active_count = poll(fds.data(), fds.size(), -1);
```

```
    if (active_count < 0) {
```

```
        std::cerr << "Poll error\n";
```

```
        break;
```

```
    }
```

```
    // 2. Iterate through our list to find out WHICH sockets woke us up
```

```
    for (size_t i = 0; i < fds.size(); ++i) {
```

```
        // revents (returned events) will be POLLIN if data is ready
```

```
        if (fds[i].revents & POLLIN) {
```

```
            if (fds[i].fd == server_fd) {
```

```
                // CASE A: The Server Socket is active. This means a NEW client is connecting!
```

```
                int new_client = accept(server_fd, NULL, NULL);
```

```
                make_socket_non_blocking(new_client);
```

```
                fds.push_back({new_client, POLLIN, 0});
```

```
                std::cout << "New client connected: " << new_client << "\n";
```

```
            }
```

```
            else {
```

```
                // CASE B: An existing client socket is active. They sent data!
```

```
                int client_fd = fds[i].fd;
```

```
                // Read the data (guaranteed not to block because poll said it's ready)
```

```

char buffer[1024] = {0};

int bytes = read(client_fd, buffer, sizeof(buffer));

if (bytes <= 0) {
    std::cout << "Client " << client_fd << " disconnected.\n";
    close(client_fd);
    // Remove from our watch list (using standard vector erase)
    fds.erase(fds.begin() + i);
    i--; // Adjust index after erase
} else {
    std::cout << "Received from " << client_fd << ": " << buffer;
}
}
}
}
}
}

```

Day 4 & 5: The Reactor Pattern

You now have two completely different architectures:

1. **The Thread Pool (Phase 2):** Great for doing heavy CPU math without blocking the server.
2. **The Event Loop (poll):** Great for managing 10,000 lazy network connections on a single thread.

The Reactor Pattern is the holy grail of system design. It combines both.

- The **Main Thread** runs the `poll()` Event Loop. It manages all 10,000 connections.
- When `poll()` says Client 42 has sent a full HTTP request, the Main Thread reads the text.
- The Main Thread then packages that text and **submits** it to the **Thread Pool** as a task.
- A Worker Thread processes the heavy logic, constructs the HTTP response, and writes it back to the client.

This means 1 Main Thread can handle 10,000 idle connections, and 4 Worker Threads can execute the actual business logic for the users who are actively making requests.

The Weekend Assignment: "The High-Concurrency Chat Room"

We are going to build a system similar to Discord's text backend or a multiplayer game server. A system where clients stay connected indefinitely.

Your Task:

Build a TCP Chat Server using `poll()` and non-blocking sockets.

1. **No Thread Pool Required Yet:** Do this entirely on the main thread using the `poll()` loop shown above.
2. **Broadcast Logic:** When Client A sends a message (e.g., "Hello everyone"), your server must iterate through the `fds` vector and `write()` that message to *all* other connected clients.
3. **Stress Test:** Connect to your server using 5 different Netcat (`nc localhost 8080`) terminals. Type a message in one, and watch it instantly appear in the other 4.

If you can successfully build this event loop, you have mastered the foundational mechanics of **Distributed Systems and Networking**.